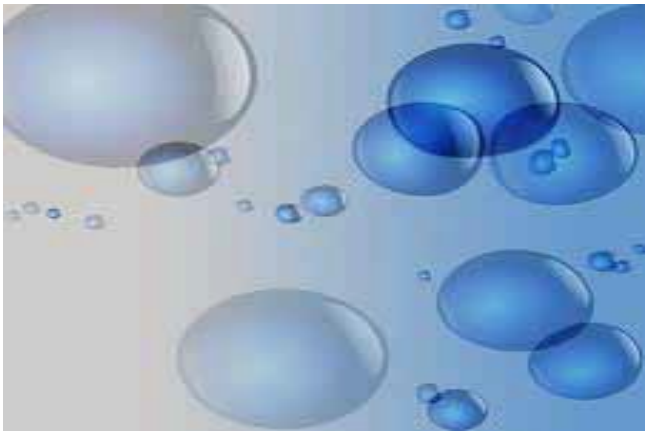
Bài 7: SẮP XẾP



Mục tiêu
Sau khi học bài này, các bạn có thể: • Mô tả đúng khái niệm, bản chất và mục đích của việc sắp xếp. • Trình bày và thực hiện cài đặt một cách chính xác các thuật toán sắp xếp bao gồm các thuật toán sắp xếp cơ bản, các thuật toán sắp xếp theo phương pháp chia để trị, Heap Sort ... • Đánh giá đúng về các thuật toán sắp xếp và tìm ra thuật toán sắp xếp phù hợp cho từng bài toán.

Nội dung
• Giới thiệu về bài toán sắp xếp. • Một số phương pháp sắp xếp cơ bản. • Các phương pháp sắp xếp theo kiểu chia để trị: Quick Sort và Merge Sort. • Khối (Heap) và Heap Sort

Thời lượng học
10 tiết

AUM VIỆT NAM

7.1. Giới thiệu về bài toán sắp xếp

Trong thực tế cuộc sống, chúng ta gặp nhiều bài toán thống kê, trong đó chúng ta cần sắp xếp thứ hạng của nhiều tổng thể khi quan sát và thu thập số liệu của một số tin tức liên quan. Chẳng hạn, trong thế vận hội Bắc Kinh năm 2008, có trên 200 đoàn tham gia thi đấu 35 môn thể thao để tranh 3 loại huy chương: vàng, bạc và đồng. Trong 18 ngày thi đấu, chúng ta được ban tổ chức công bố thứ hạng của các đoàn. Thực chất, đó là một bài toán sắp xếp thống kê. Trong nhiều ứng dụng của tin học, yêu cầu về sắp xếp thường xuất hiện với các mục đích khác nhau: sắp xếp dữ liệu trong máy tính để thuận lợi cho việc tìm kiếm xử lý để in bảng, biểu v.v…

Sắp xếp là quá trình xử lý một danh sách các phần tử (hoặc các mẫu tin) để đặt chúng theo một thứ tự, thỏa mãn một tiêu chuẩn nào đó dựa trên nội dung thông tin lưu giữ tại mỗi phần tử.

Cho trước một dãy số $\mathbf{a_1, a_2,…, a_N}$ được lưu trữ trong cấu trúc dữ liệu mảng

Sắp xếp dãy số $\mathbf{a_1, a_2,…, a_N}$ là thực hiện việc bố trí lại các phần tử sao cho được dãy mới $\mathbf{a_{k1}, a_{k2},…, a_{kN}}$ được hình thành có thứ tự (giả sử xét thứ tự tăng) nghĩa là $a_{ki} > a_{ki-1}$... Hai thao tác so sánh và gán là các thao tác cơ bản của hầu hết các thuật toán sắp xếp. Bài toán sắp xếp xuất hiện trong bất kỳ lĩnh vực nào của tin học, từ những ứng dụng ẩn bên trong của hệ điều hành như bài toán điều khiển quá trình, bài toán lập lịch cho CPU, bài toán quản lý bộ nhớ… cho đến những ứng dụng thông thường như sắp xếp dãy các từ, các câu, các bản ghi theo thứ tự nào đó.

Khi xây dựng một thuật toán sắp xếp, cần tìm cách giảm thiểu những phép so sánh và đổi chỗ không cần thiết để tăng hiệu quả của thuật toán. Đối với các dãy số được lưu trữ trong bộ nhớ chính, nhu cầu tiết kiệm bộ nhớ được đặt nặng, do vậy những thuật toán sắp xếp đòi hỏi cấp phát thêm vùng nhớ để lưu trữ dãy kết quả ngoài vùng nhớ lưu trữ dãy số ban đầu thường ít được quan tâm. Thay vào đó, các thuật toán sắp xếp trực tiếp trên dãy số ban đầu – gọi là các thuật toán sắp xếp tại chỗ – lại được đầu tư phát triển. Phần này, giới thiệu một số giải thuật sắp xếp từ đơn giản đến phức tạp có thể áp dụng thích hợp cho việc sắp xếp.

Sau đây là một số phương pháp sắp xếp thông dụng sẽ được đề cập đến trong bài này:

- Sắp xếp kiểu chọn – Selection Sort.
- Sắp xếp kiểu chèn – Insertion Sort.
- Sắp xếp kiểu nổi bọt – Bubble Sort.
- Sắp xếp nhanh – Quick Sort.
- Sắp xếp kiểu trộn – Merge Sort.
- Sắp xếp kiểu khối – Heap Sort.

Chúng ta sẽ lần lượt khảo sát các thuật toán trên. Các thuật toán như Insertion Sort, Selection Sort, Bubble Sort là những thuật toán đơn giản dễ cài đặt nhưng chi phí cao. Các thuật toán Heap Sort, Quick Sort, Merge Sort phức tạp hơn nhưng hiệu suất cao hơn nhóm các thuật toán đầu. Cả hai nhóm thuật toán trên đều có một điểm chung là được xây dựng dựa trên cơ sở so sánh giá trị của các phần tử trong mảng (hay so sánh các khóa tìm kiếm).

7.2.1. Sắp xếp kiểu chèn – Insertion Sort

AUM VIỆT NAM

7.2.1.1. Tư tưởng của thuật toán

Giả sử có một dãy $a_1, a_2,\ldots, a_n$, trong đó i là phần tử đầu tiên của phần đầu dãy $a_1, a_2,\ldots, a_{i-1}, a_i$ đã có thứ tự. Ý tưởng của thuật toán là chèn phần tử thứ i + 1 vào i phần tử đầu tiên của dãy đã có thứ tự để được dãy mới $a_1, a_2,\ldots, a_{i+1}$ trở nên có thứ tự. Việc chèn được thực hiện như sau: tiến hành tìm vị trí của phần tử i + 1 trong k phần tử đầu tiên bằng cách vận dụng thuật giải tìm kiếm (tuần tự hoặc nhị phân), sau khi tìm được vị trí chèn thì chúng ta sẽ tiến hành chèn phần tử i + 1 bằng cách dời các phần tử từ vị trí chèn đến phần tử thứ i sang phải 1 vị trí và chèn phần tử i + 1 vào vị trí đó.

7.2.1.2. Nội dung và cách cài đặt của thuật toán

Thuật toán này sắp xếp một dãy $a_1, a_2,\ldots, a_n$ theo thứ tự tăng dần bằng phương pháp sắp xếp kiểu chèn.

```
Bước 1: i = 1;
Bước 2:đặt
       x = a[i];
       j = i - 1;
Bước 3: while (j >= 0) && (x < a[j])
               a[j + 1] = a[j];
               j--;
Bước 4: đặt a[j + 1] = x ;
       i++;
Bước 5: nếu i < = n - 1 lặp lại bước 2 ngược lại thì kết thúc
```

Cài đặt thuật toán:

```
#define  Max_Size …
typedef  Kieu_du_lieu KeyType;
         //kiểu dữ liệu của các khóa sắp xếp
typedef  struct KeyArray
                {
                  KeyType Array[Max_Size];
                  //mảng các phần tử có kiểu KeyType
                  int n;
                  //số lượng phần tử của mảng
                };
KeyArray Sortinsert( KeyArray a)
{
   int i,j;
   KeyType x;
   i = 1;
   while ( i <= a.n - 1 )
```

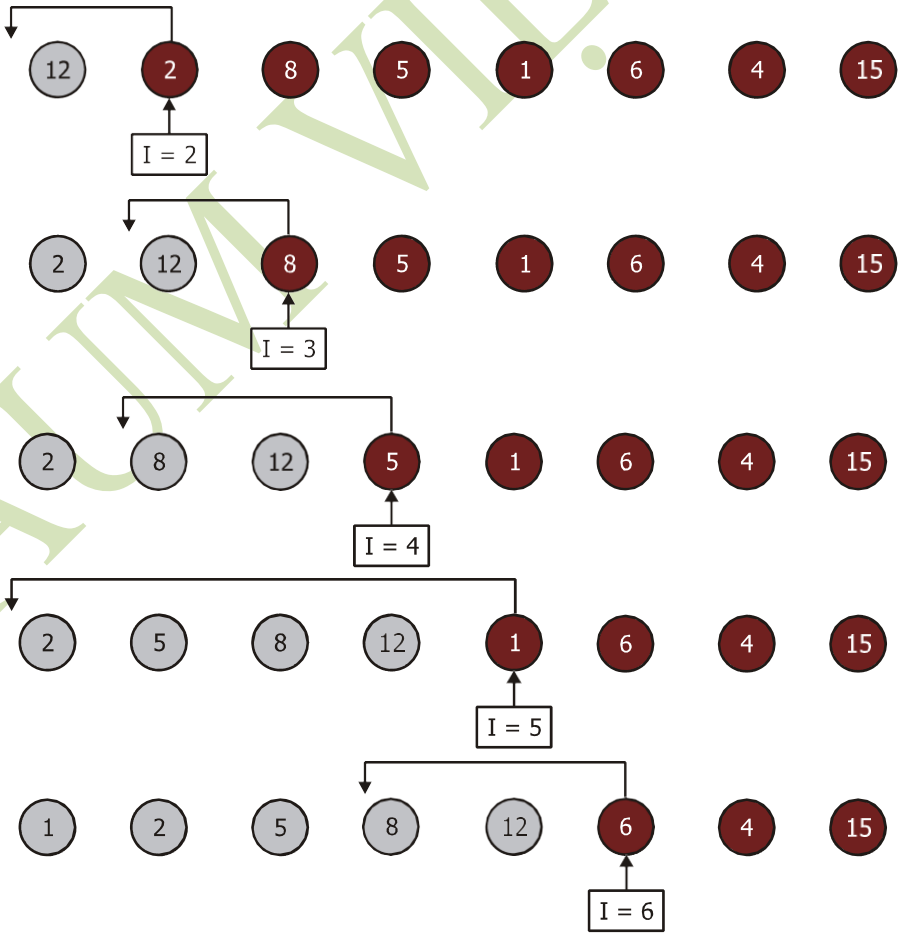
AUM VIỆT NAM

```
            {
                        x = a.Array[i] ; j = i - 1;
                        while (( j >= 0 )&&(x < a.Array[j]))
                                    {
                                a.Array[j + 1] = a.Array[j];
                                            j--;
                                    }
                        a.Array[j + 1] = x ;
                        //đưa giá trị cần chèn vào vị trí trống
                        i++;
            }
    return a;
}
```

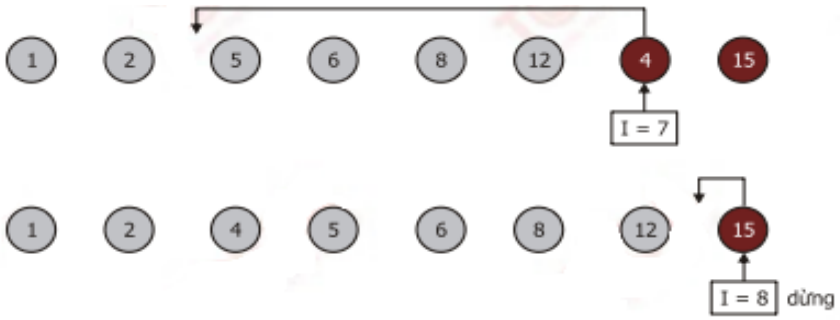
Trong cách cài đặt thuật toán sắp xếp chèn ở trên, ta đã sử dụng phương pháp tìm kiếm tuần tự để xác định vị trí cần chèn. Do đoạn a[1] đến a[i – 1] đã được sắp xếp, nên ta hoàn toàn có thể sử dụng giải thuật tìm nhị phân để thực hiện việc tìm vị trí để chèn a[i] vào. Việc xây dựng giải thuật sắp xếp kiểu chèn với phương pháp tìm kiếm nhị phân để xác định vị trí chèn xem như là bài tập cho học viên tự giải.

Ví dụ 7.1. Cho dãy số a: 12, 2, 8, 5, 1, 6, 4, 15

Sắp xếp dãy số a này theo thứ tự tăng dần bằng phương pháp sắp xếp chèn. Ta thực hiện như sau:



AUM VIỆT NAM



Đánh giá thuật toán

Đối với giải thuật sắp xếp kiểu chèn, các phép so sánh xảy ra trong mỗi vòng lặp **while** là tìm vị trí chèn thích hợp **j**. Mỗi lần xác định vị trí đang xét không thích hợp, sẽ dời chỗ phần tử a[j] tương ứng. Giải thuật thực hiện tất cả N – 1 vòng lặp **while**. Do số lượng phép so sánh và phép dời chỗ này phụ thuộc vào tình trạng của dãy số ban đầu, nên ta chỉ có thể ước lược trong từng trường hợp như sau:

- Trường hợp tốt nhất, khi dãy phần tử cần sắp xếp có thứ tự tăng:
  - Số phép gán: $S_{gán} = 2(n - 1)$
  - Số phép so sánh: $S_{so\ sánh} = n - 1$
- Trường hợp xấu nhất, khi dãy phần tử cần sắp xếp có thứ tự giảm dần:
  - Số phép gán: $S_{gán} = \frac{n(n+1)}{2} - 1$
  - Số phép so sánh: $S_{so\ sánh} = \frac{n(n-1)}{2}$

AUM VIỆT NAM

Như vậy, độ phức tạp của thuật toán sắp xếp chèn là $O(n^2)$ cho cả trường hợp xấu nhất và trường hợp trung bình.

7.2.2. Sắp xếp kiểu chọn – Selection sort

7.2.2.1. Tư tưởng thuật toán

Ban đầu, dãy gồm n phần tử không có thứ tự. Ta chọn phần tử nhỏ nhất trong n phần tử của dãy cần sắp xếp và đổi giá trị của nó với a[1], khi đó a[1] có giá trị nhỏ nhất trong dãy.

Lần thứ 2, chọn phần tử nhỏ nhất trong n – 1 phần tử còn lại a[2], a[3], ..., a[n] và đổi giá trị của nó với a[2].

Ở lượt thứ i ta chọn trong dãy a[i ... n] ra phần tử nhỏ nhất và đổi chỗ giá trị của nó với a[i].

Tới lượt thứ n – 1. Chọn trong 2 phần tử a[n – 1] và a[n] phần tử có giá trị nhỏ hơn và đổi giá trị của nó với a[n – 1]. Khi đó, dãy thu được có thứ tự không giảm.

7.2.2.2. Nội dung và cách cài đặt thuật toán

Thuật toán sắp xếp n phần tử trong mảng a[1], a[2], ..., a[n] theo thứ tự tăng dần

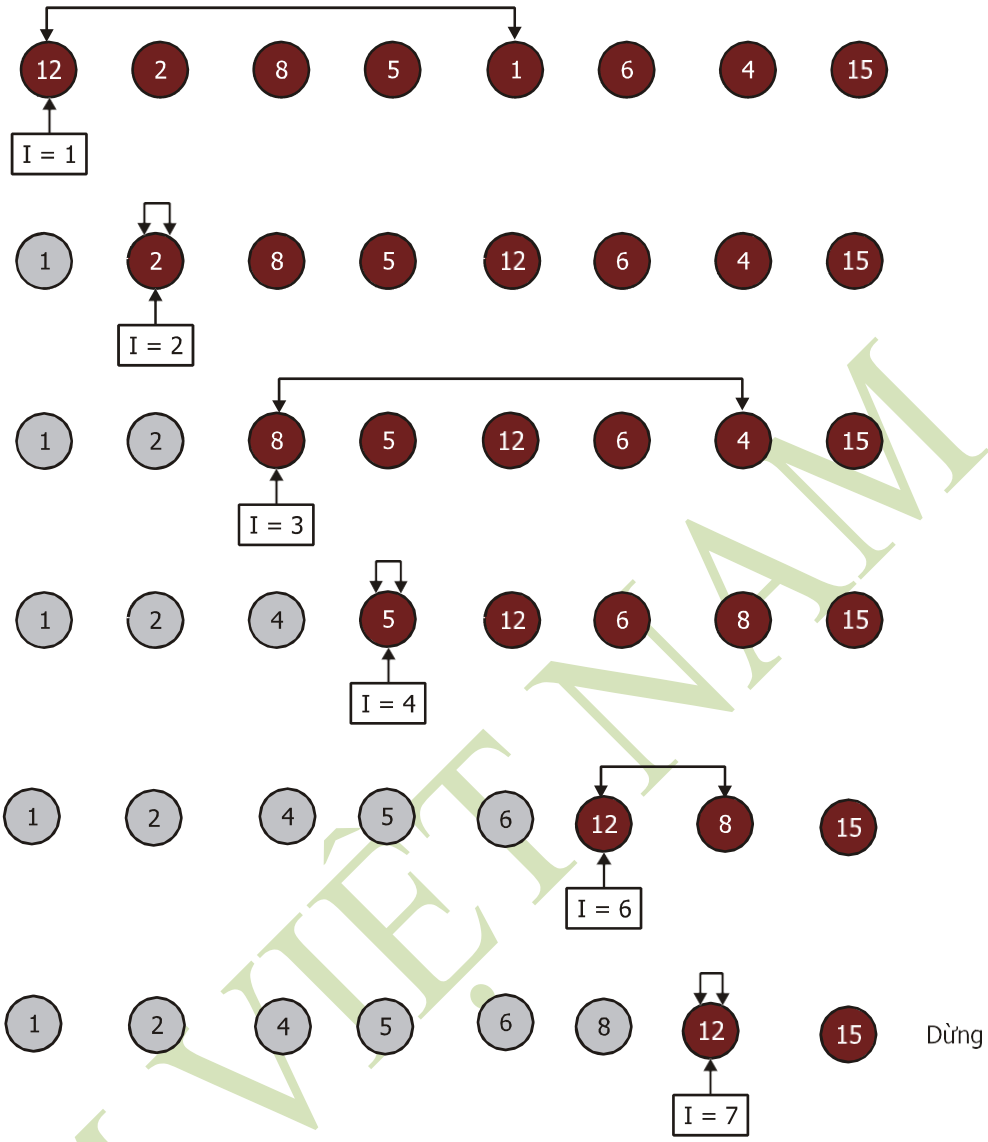
```
Bước 1: i = 0;
Bước 2: tìm phần tử a[min] nhỏ nhất trong dãy từ a[i] tới a[n – 1]
Bước 3: hoán vị a[min] và a[i]
Bước 4 : nếu i <= n – 2 thì i = i + 1, lặp lại bước 2, ngược lại thì dừng
```

Cài đặt thuật toán:

```
KeyArray SortSelect(KeyArray a)
{
    int i,j,min;
    KeyType tg;
    i = 0;
    for (i = 0; i < a.n – 2; i++)
                    {
                        min = i;
                        for (j = i + 1; j < a.n – 1; j++)
                                if (a.Array[min] > a.Array[j])
                                min = j;
                        tg = a.Array[i];
                        a.Array[i] = a.Array[min];
                        a.Array[min] = tg;
                    }
    return a;
}
```

Ví dụ 7.2. Cho dãy số a: 12, 2, 8, 5, 1, 6, 4, 15. Sắp xếp dãy số này theo thứ tự tăng dần bằng thuật toán sắp xếp lựa chọn được minh họa như sau:

AUM VIỆT NAM


12 2 8 5 1 6 4 15
I = 1
1 2 8 5 12 6 4 15
I = 2
1 2 8 5 12 6 4 15
I = 3
1 2 4 5 12 6 8 15
I = 4
1 2 4 5 6 12 8 15
I = 6
1 2 4 5 6 8 12 15
I = 7
Dừng

AUM VIỆT NAM

Đánh giá thuật toán

Ta nhận thấy ở lượt thứ I, để chọn ra phần tử nhỏ nhất bao giờ cũng cần n – i phép so sánh. Số lượng phép so sánh trong thuật toán này không phụ thuộc vào tình trạng ban đầu của mảng, do đó, tổng số phép toán so sánh là:

$$S_{so\ sánh} = (n - 1) + (n - 2) + ... + 2 + 1 = n(n - 1) / 2$$

- Trong trường hợp tốt nhất, khi dãy đã có thứ tự tăng dần: Số phép gán $S_{gán} = 0$
- Trong trường hợp tồi nhất khi dãy có thứ tự giảm dần:

$$\text{Số phép gán } S_{gán} = 3 * (n - 1) + n * (n - 1) / 2 = (n - 1)(n + 6) / 2$$

Như vậy, độ phức tạp của thuật toán sắp xếp lựa chọn này là $O(n^2)$.

7.2.3. Sắp xếp kiểu đổi chỗ

Các thuật toán trong kiểu sắp xếp này sẽ tìm cách đổi chỗ các phần tử đứng sai vị trí (so với dãy đã sắp xếp) trong dãy a cho nhau để cuối cùng tất cả các phần tử trong dãy a đều về đúng vị trí như dãy đã sắp xếp. Trong phần này, chúng ta sẽ đi tìm hiểu một thuật toán khá đơn giản, dễ hiểu và dễ cài đặt cho kiểu sắp xếp đổi chỗ đó là thuật toán sắp xếp nổi bọt (bubble Sort).

7.2.3.1. Tư tưởng thuật toán sắp xếp nổi bọt

Đi từ cuối mảng về đầu mảng, trong quá trình đi, nếu phần tử ở dưới (đứng phía sau) nhỏ hơn phần tử đứng ngay trên (trước) nó thì theo nguyên tắc của bọt khí, phần tử nhẹ hơn sẽ bị "trồi" lên trên phần tử nặng (hai phần tử này sẽ được đổi chỗ cho nhau). Kết quả là phần tử nhỏ nhất (nhẹ nhất) sẽ được đưa về đầu dãy rất nhanh. Sau đó sẽ không xét đến nó ở bước tiếp theo, do vậy ở lần xử lý thứ i sẽ có vị trí đầu dãy là i. Lặp lại xử lý trên cho đến khi không còn cặp phần tử nào để xét và ta có dãy sắp xếp theo một thứ tự.

7.2.3.2. Nội dung và cách cài đặt thuật toán

Thuật toán này sắp xếp kiểu nổi bọt một danh sách gồm các mục a[1], a[2], ..., a[n] theo thứ tự tăng dần. Thuật toán được thể hiện như sau:

```
KeyType tg;
Bước 1: i = 0;
        for (i = 0;i < n - 2;i++)//lần lượt xử lý với mỗi i
Bước 2: duyệt từ cuối ngược về vị trí i
        for (j = n - 1; j >= i + 1; j--)
          if (a[j - 1] >a [j])//hoán vị trí a[j] và a[j - 1]
                        {
                                tg = a[j - 1];
                                a[j - 1] = a[j];
                                a[j] = tg;
                        }
```

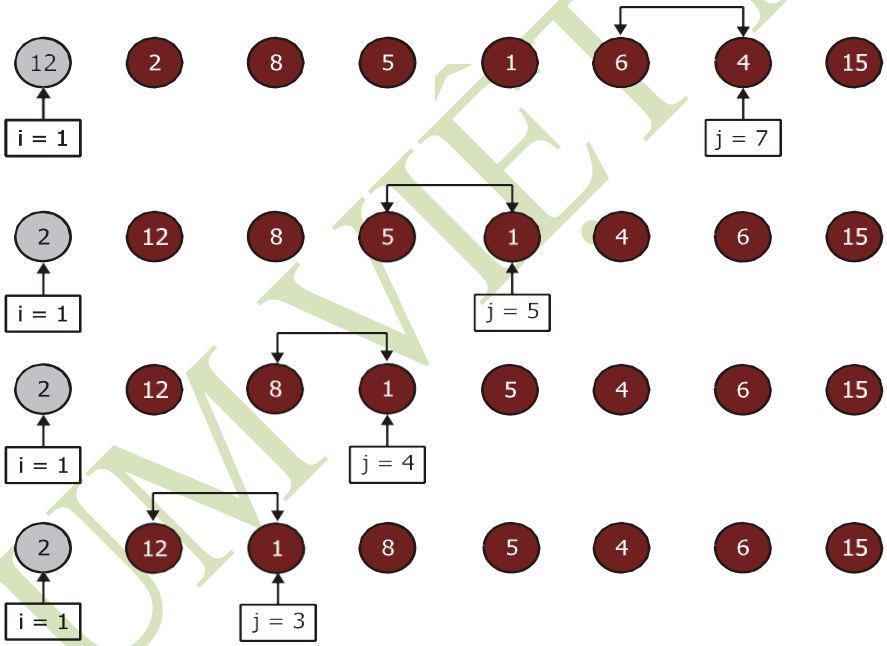
AUM VIỆT NAM

Cài đặt thuật toán:

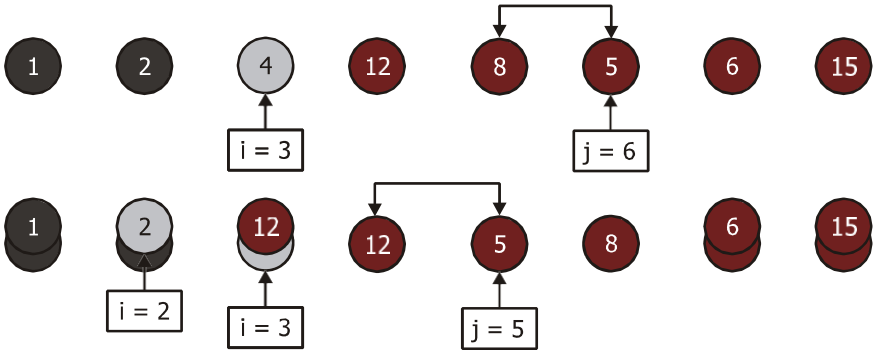
```
KeyArray Sortbubble(KeyArray a)
  {
      int i,j;
      KeyType tg;
      i = 0;
      for (i = 0; i < a.n - 2; i++)
                    for (j = a.n - 1; j >= i + 1; j--)
                                  if (a.Array[j -1 ] > a.Array[j])
                                   {
                                   tg = a.Array[j - 1];
                                         a.Array[j - 1] = a.Array[j];
                                         a.Array[j] = tg;
                                   }
       return a;
  }
```

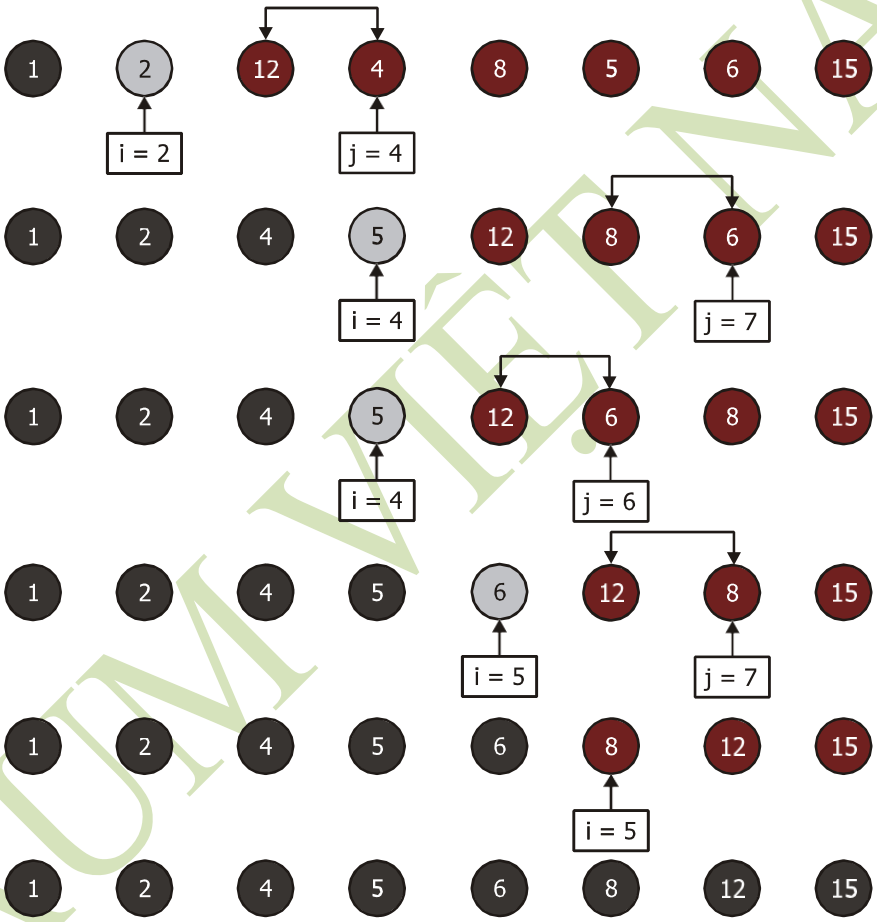
Ví dụ 7.3. Cho dãy số a: 12, 2, 8, 5, 1, 6, 4, 15 .

Sắp xếp dãy số a theo thứ tự tăng dần bằng thuật toán sắp xếp nổi bọt được minh họa dưới đây:



AUM VIỆT NAM


1 2 4 12 8 5 6 15
i = 3
j = 6
1 2 12 12 5 8 6 15
i = 2
i = 3
j = 5


1 2 12 4 8 5 6 15
i = 2
j = 4
1 2 4 5 12 8 6 15
i = 4
j = 7
1 2 4 5 12 6 8 15
i = 4
j = 6
1 2 4 5 6 12 8 15
i = 5
j = 7
1 2 4 5 6 8 12 15
i = 5
1 2 4 5 6 8 12 15

AUM VIỆT NAM

7.2.3.3. Đánh giá thuật toán

Ta nhận thấy trong giải thuật nổi bọt, số lượng các phép so sánh không phụ thuộc vào trạng thái ban đầu của dãy cần sắp xếp, nhưng số hoán vị lại phụ thuộc vào trạng thái ban đầu của dãy. Trong trường hợp tốt nhất, tức là dãy ban đầu đã có thứ tự tăng dần thì tổng số hoán vị $S_{hv} = 0$.

Trong trường hợp xấu nhất, khi danh sách có thứ tự giảm dần: ta thấy, khi đi qua dãy lần đầu có n – 1 lần so sánh và n – 1 lần hoán vị. Trong lần thứ 2, dãy con gồm n – 1 phần tử được duyệt và có n – 2 lần so sánh và hoàn vị. Quá trình này tiếp tục cho đến khi danh sách con chỉ gồm 2 phần tử và trong lần này có một lần so sánh và hoán vị. Như vậy, trong trường hợp xấu nhất:

Số lần so sánh là $S_{so\ sánh} = (n – 1) + (n – 2) + ... + 2 + 1 = n(n – 2) / 2$

Số lần hoán vị là $S_{hoán\ vị} = (n – 1) + (n – 2) + ... + 2 + 1 = n(n – 2) / 2$

Vì các lệnh so sánh và hoán vị là những lệnh thường xuất hiện nhất trong thuật toán này nên chúng xác định độ phức tạp của thuật toán. Do vậy, độ phức tạp của thuật toán này trong trường hợp xấu nhất là $O(n^2)$.

7.3. Các phương pháp sắp xếp theo kiểu chia để trị: Quick Sort và Merge Sort

7.3.1. Tư tưởng chính của các phương pháp sắp xếp theo kiểu chia để trị

Từ những giải thuật đã được trình bày trên và từ kinh nghiệm thực tế, ta nhận thấy việc sắp xếp danh sách dài thì khó hơn là sắp xếp danh sách ngắn. Nếu chiều dài danh sách tăng gấp đôi thì công việc sắp xếp thông thường tăng hơn gấp đôi (với sắp xếp kiểu chèn và sắp xếp kiểu chọn, khối lượng công việc tăng lên khoảng bốn lần). Do đó, nếu có thể chia một danh sách ra thành hai phần có kích thước xấp xỉ nhau và thực hiện việc sắp xếp mỗi phần một cách riêng rẽ thì khối lượng công việc cần cho việc sắp xếp sẽ giảm đi đáng kể.

Ví dụ, việc sắp xếp các phiếu trong thư viện sẽ nhanh hơn nếu các phiếu được chia thành từng nhóm có chung chữ cái đầu và từng nhóm được tiến hành sắp xếp riêng rẽ. Ở đây, chúng ta vận dụng ý tưởng chia một bài toán thành nhiều bài toán tương tự như bài toán ban đầu nhưng nhỏ hơn và giải quyết các bài toán nhỏ này. Sau đó, chúng ta tổng hợp lại để có lời giải cho toàn bộ bài toán ban đầu. Phương pháp này được gọi là "chia để trị" (*Divide – and – Conquer*).

Để sắp xếp các danh sách con, chúng ta lại áp dụng chiến lược chia để trị để tiếp tục chia nhỏ từng danh sách con. Quá trình này dĩ nhiên không bị lặp vô tận. Khi ta có các danh sách con với kích thước là 1 phần tử thì quá trình dừng.

Vấn đề còn lại cần xem xét là cách phân hoạch (*Partition*) danh sách ban đầu và cách kết nối (*Combine*) hai danh sách đã có thứ tự cho thành một danh sách có thứ tự. Do đó, chúng ta sẽ đi nghiên cứu hai phương pháp sắp xếp là Quick Sort và Merge Sort, mỗi phương pháp sẽ làm việc tốt ứng với một số trường hợp riêng.

7.3.2. Quick Sort

7.3.2.1. Tư tưởng của thuật toán

Trong các thuật toán sắp xếp đã tìm hiểu trước đây, ý tưởng cơ bản là chọn phần tử nhỏ nhất hay lớn nhất trong một danh sách con nào đó của danh sách và đặt nó vào đúng vị trí trong danh sách con. Trong sơ đồ thuật toán này, nó cũng chọn một mục và

AUMVIETNAM

định vị đúng cho mục đó. Tuy nhiên, mục được chọn không nhất thiết là phần tử nhỏ nhất hay lớn nhất mà là một phần tử ngẫu nhiên của danh sách cần sắp xếp làm khóa chốt. Tính từ khóa chốt, các phần tử nhỏ hơn khóa chốt phải được xếp trước chốt; mọi phần tử lớn hơn khóa chốt được xếp vào sau chốt. Để làm được điều đó, các phần tử trong danh sách sẽ được so sánh với khóa chốt và tráo vị trí cho nhau hoặc cho khóa chốt nếu phần tử đó lớn hơn chốt mà lại nằm trước chốt hoặc nhỏ hơn chốt nhưng lại nằm sau chốt. Khi việc đổi chỗ lần đầu tiên đã được thực hiện xong thì danh sách tạo thành hai đoạn: một đoạn gồm các phần tử nhỏ hơn chốt, một đoạn gồm các phần tử lớn hơn chốt còn khóa chốt chính là vị trí của phần tử trong danh sách được sắp xếp. Áp dụng kỹ thuật như trên cho mỗi đoạn trước chốt và sau chốt cho tới khi các đoạn còn lại hai phần tử thì dừng lại. Khi đó, danh sách đã được sắp xếp.

Cụ thể giả sử để sắp xếp dãy số a gồm n phần tử: a_1, a_2,…, a_n; giải thuật Quick Sort dựa trên việc phân hoạch dãy ban đầu được chia thành hai phần:

Dãy con 1 gồm các phần tử: a_1,…, a_{i-1} có giá trị nhỏ hoặc bằng a_i;

Dãy con 2 gồm các phần tử: a_{i+1},..., a_n có giá trị lớn hơn a_i;

Với a_i là một phần tử bất kỳ trong dãy.

Để sắp xếp dãy con 1 và 2, lần lượt tiến hành việc phân hoạch từng dãy con theo cùng phương pháp phân hoạch dãy ban đầu.

7.3.2.2. Nội dung và cách cài đặt thuật toán

```
KeyType Item ,X;
Bước 1 : first = 0;
Bước 2 : last = n - 1;
Bước 3:Xuất phát từ đầu dãy  để tìm phần tử có giá trị > X
       i = first;
Bước 4: Xuất phát từ cuối dãy để tìm phần tử có giá trị < X
       j = last;
Bước 5: Khóa chốt là phần tử giữa
        X = a[(i + j)/ 2];
Bước 6:Lặp tìm phần tử có giá trị lớn hơn X và nhỏ hơn X
 do
{
                while (a[i] < X)
                                i = i + 1;
                while (a[j] > X)
                                j = j - 1;
                if (i <= j)
                  {
                    //hoán vị a[i] và a[j]
                    Item = a[i];
                    a[i] = a[j];
                    a[j] = Item;
                    i = i + 1;
                    j = j - 1;
                  }
```

AUM VIỆT NAM

```
}
while (i <= j);
Bước 7: Phân hoạch đệ quy dãy con từ phần tử thứ First đến phần tử thứ j
  a = Quicksort(a, i, last);
Bước 8: Phân hoạch đệ quy dãy con từ phần tử thứ i đến phần tử thứ Last
a = Quicksort(a, first, j);
```

Cài đặt thuật toán:

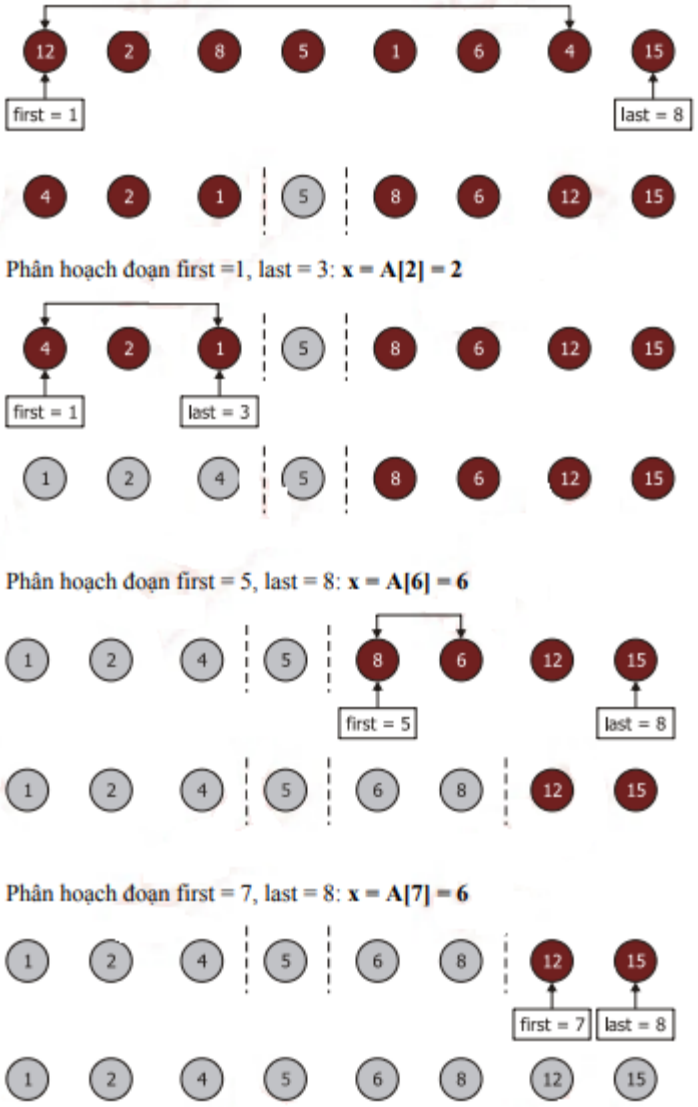
```
KeyArray Quicksort(KeyArray a,int first, int last)
{
            int i, j;
            KeyType Item ,X;
            if  (first > last)
                        printf("day con khong co phan tu");
            i = first;
            j = last;
            X = a.Array[(i + j)/ 2];
            do
                  {
                        while (a.Array[i] < X)
                                    i = i + 1;
                        while (a.Array[j] > X)
                                    j = j - 1;
                        if (i <= j)
                         {
                              Item = a.Array[i];
                              a.Array[i] = a.Array[j];
                              a.Array[j] = Item;
                              i = i + 1;
                              j = j - 1;
                         }
                   }
             while (i <= j);
             a = Quicksort(a, i, last);
             a = Quicksort(a, first, j);
             return a;
}
```

Ví dụ 7.4. Cho dãy số a: 12, 2, 8, 5, 1, 6, 4, 15

Dãy a được sắp xếp theo thứ tự tăng dần bằng thuật toán sắp xếp Quick Sort như sau:

Phân hoạch đoạn first = 1, last = 8: **x = A[4] = 5**

AUM VIỆT NAM



7.3.2.3. Đánh giá thuật toán

Hiệu quả thực hiện của giải thuật Quick Sort phụ thuộc vào việc chọn khóa chốt để phân đoạn. Trường hợp tốt nhất xảy ra nếu mỗi lần phân đoạn đều chọn được khóa chốt lớn hơn (hay bằng) nửa số phần tử, và nhỏ hơn (hay bằng) nửa số phần tử còn lại; khi đó độ phức tạp tính toán của Quick Sort là O(nlgn). Nhưng nếu mỗi lần chọn khóa chốt để phân đoạn lại chọn nhằm phần tử có giá trị cực đại (hay cực tiểu) là khóa chốt, dãy sẽ bị phân chia thành 2 phần không đều: một phần chỉ có 1 phần tử, phần còn lại gồm (n – 1) phần tử. Do vậy, cần n lần phân đoạn mới sắp xếp xong. Khi đó, độ phức tạp tính toán của Quick Sort là $O(n^2)$. Thời gian thực hiện giải thuật Quick Sort trung bình là O(nlgn).

- Về nguyên tắc, có thể chọn khóa chốt để phân đoạn là một phần tử tùy ý trong dãy, nhưng để đơn giản, dễ diễn đạt giải thuật, phần tử có vị trí giữa thường được chọn.
- Việc chọn chốt cho phép phân đoạn quyết định hiệu quả của thuật toán. Nếu chọn chốt không tốt, rất có thể việc phân đoạn bị suy biến thành trường hợp xấu nhất khiến Quick Sort hoạt động chậm và tràn ngăn xếp chương trình con khi gặp phải dây chuyền đệ quy quá dài. Cụ thể:
  - Nếu chọn khóa chốt là phần tử đầu tiên hay phần tử cuối cùng của danh sách thì độ phức tạp của Quick Sort là xấu nhất với danh sách được sắp xếp theo thứ tự từ phần tử đầu tiên đến phần tử cuối cùng.
  - Nếu chọn khóa chốt là phần tử ở giữa danh sách thì Quick Sort có độ phức tạp tồi nhất với danh sách được sắp xếp bắt đầu từ phần tử ở giữa danh sách trở về hai đầu của danh sách.
  - Nếu chọn khóa chốt là phần tử ở vị trí ngẫu nhiên trong danh sách thì khó tìm ra bộ dữ liệu, khiến cho Quick Sort suy biến. Mặc dù trong n! hoán vị (giả sử danh sách gồm n phần tử) của danh sách thế nào cũng có một dãy làm Quick Sort suy biến. Tuy nhiên, xác suất xảy ra danh sách này là quá nhỏ và rất khó để chỉ ra nên việc chọn khóa chốt là là phần tử nằm ở vị trí ngẫu nhiên có thể coi là an toàn với các trường hợp suy biến của Quick Sort.
- Để Quick Sort có độ phức tạp tính toán tốt nhất, ta phải tìm được khóa chốt là phần tử median (phần tử trung vị). Tuy nhiên, chi phí để tìm ra phần tử median là rất lớn nên trong nhiều trường hợp sắp xếp, ta chọn phần tử ở chính giữa hoặc phần tử ngẫu nhiên làm khóa chốt với hi vọng nó gần phần tử median nhất.

7.3.3. Merge Sort

7.3.3.1. Tư tưởng của thuật toán

Để sắp xếp dãy $a_1, a_2,\ldots, a_n$, giải thuật Merge Sort dựa trên nhận xét sau:

Mỗi dãy $a_1, a_2,\ldots, a_n$ bất kỳ đều có thể coi như là một tập hợp các dãy con liên tiếp mà mỗi dãy con đều đã có thứ tự. Ví dụ, dãy 12, 2, 8, 5, 1, 6, 4, 15 có thể coi như gồm 5 dãy con không giảm (12); (2, 8); (5); (1, 6); (4, 15).

Dãy đã có thứ tự coi như 1 dãy con.

Như vậy, một cách tiếp cận để sắp xếp dãy là tìm cách làm giảm số dãy con không giảm của nó. Đây chính là hướng tiếp cận của thuật toán sắp xếp theo phương pháp trộn. Trong phương pháp Merge Sort, mấu chốt của vấn đề là cách phân hoạch dãy ban đầu thành các dãy con. Sau khi phân hoạch xong, dãy ban đầu sẽ được tách ra thành

2 dãy phụ theo nguyên tắc phân phối đều luân phiên. Trộn từng cặp dãy con của hai dãy phụ thành một dãy con của dãy ban đầu, ta sẽ nhận lại dãy ban đầu nhưng với số lượng dãy con, ít nhất giảm đi một nửa. Lặp lại quy trình trên sau một số bước, ta sẽ nhận được 1 dãy chỉ gồm 1 dãy con không giảm, nghĩa là dãy ban đầu đã được sắp xếp.

Các thuật toán sắp xếp bằng phương pháp trộn bao gồm:

- Thuật toán sắp xếp trộn thẳng hay trộn trực tiếp (Straight Merge Sort),
- Thuật toán sắp xếp trộn tự nhiên (Natural Merge Sort).

7.3.3.2. Thuật toán sắp xếp trộn trực tiếp

- **Tư tưởng:**

  Ban đầu, việc phân hoạch thành các dãy con đơn giản chỉ là tách dãy gồm n phần tử thành n dãy con. Đòi hỏi của thuật toán về tính có thứ tự của các dãy con luôn được thỏa mãn trong cách phân hoạch này vì dãy gồm một phân tử luôn có thứ tự dãy. Sau đó, trộn tương ứng từng cặp dãy con thành các dãy con mới có chiều dài bằng 2 để đưa dãy ban đầu thành n/2 dãy con.

  Như vậy, sau mỗi lần phân phối và trộn các dãy con thì số dãy con sẽ giảm đi một nửa, đồng thời chiều dài mỗi dãy con sẽ tăng gấp đôi. Do đó, sau $Log_2(N)$ lần phân phối và trộn thì dãy ban đầu được sắp xếp có thứ tự.

- **Nội dung và cách cài đặt của thuật toán:**

  Để tiến hành xây dựng thuật toán sắp xếp trộn trực tiếp ta viết các thủ tục sau:

  - Thủ tục 1: thủ tục này trộn mảng A[a..b] với mảng A[b+1..c] để được mảng B[a..c].
  - Thủ tục 2: thủ tục này trộn lần lượt các cặp dãy con theo thứ tự.

  Cuối cùng là thủ tục Merge Sort. Thủ tục này cần một dãy khóa phụ t[1..n]. Trước hết, ta gọi MergeByLenght(k, t, 1) để trộn hai khóa liên tiếp của k thành một mạch trong t, sau đó gọi lại MergeByLenght(t, k, 2) để trộn 2 mạch liên tiếp trong t thành một mạch trong k,… Như vậy, k và t được sử dụng luân phiên: một dãy chứa các mạch và một dãy dùng để trộn các cặp mạch liên tiếp để được mạch lớn hơn.

- **Cài đặt thuật toán:**

```
void MergeSort( KeyArray A,int n)
{
        int i, j, p;
        p = a;
        i = a;
        j = b + 1;
        while ((i <= b)&&(j <= c))
          {
             if (A.Array[i] <= A.Array[j])
                 {
                      B.Array[p] = A.Array[i];
                      i++;
                  }
             else
                  {
                      B.Array[p] = A.Array[j];
                      j++;
                  }
              p = p + 1;
          }
        if (i <= b)
           {
              int tg1,tg2;
```

AUM VIỆT NAM

```
                tg1 = p; tg2
                = i;
                while((tg1 <= c)&&(tg2 <= b))
                        {
                                B.Array[tg1] = A.Array[tg2]; tg1++;
                                tg2++;

                        }
                }
            else
                {
                int tg1,tg2; tg1 =
                p;
                tg2 = j;
                    while((tg1 <= c)&&(tg2 <= c))
                        {
                            B.Array[tg1] = A.Array[tg2]; tg1++;
                            tg2++;
                        }
                }
    }
void Merge( KeyArray A,KeyArray B,int a,int                b,int c)
        {
            int a,b,c; a =
            1;
            b = len;
            c = 2 * len;
            while (c <= A.n – 1)
                {
                    Merge(A, B, a, b, c);
                                a = a + 2 * len; b = b +2
                                * len; c = c + 2 * len;
                }
             if (b < A.n–1 )
                                Merge(A, B, a, b, A.n – 1);
              else
                        if (a <= A.n – 1)
                            for (int i = a; i<A.n – 1; i++) B.Array[i] =
                                A.Array[i];
        }
void MergrSort(KeyArray K)
{
        KeyArray T; // mảng phụ
        int     len;
        int     Flag; // Flag = 1: trộn K vào T hoặc ngược lại Flag = 1;
        len = 1;
```

AUM VIỆT NAM

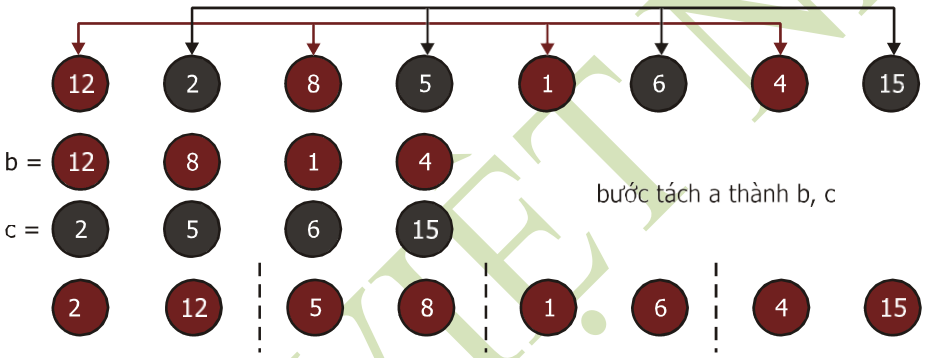
```
        while (len < K.n - 1)
                {
                        if  (Flag == 1 )
                                MergeByLenght(K, T, len);
                        else
                                MergeByLenght(T, K, len);
                        len = len * 2;
                        Flag = 0 ;
                }
        if (Flag != 1)
                         K = T;
}
```

Ví dụ 7.5.

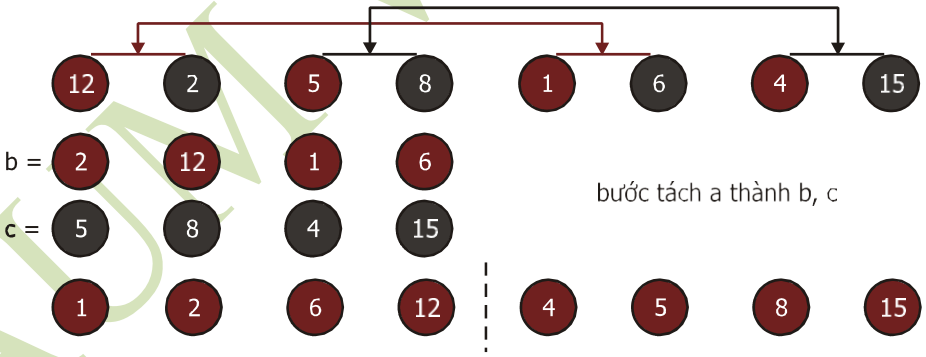
Cho dãy số a: 12 , 2, 8, 5, 1, 6, 4, 15

Sắp xếp dãy trên theo thứ tự không giảm bằng thuật toán trộn trực tiếp được minh họa như sau:

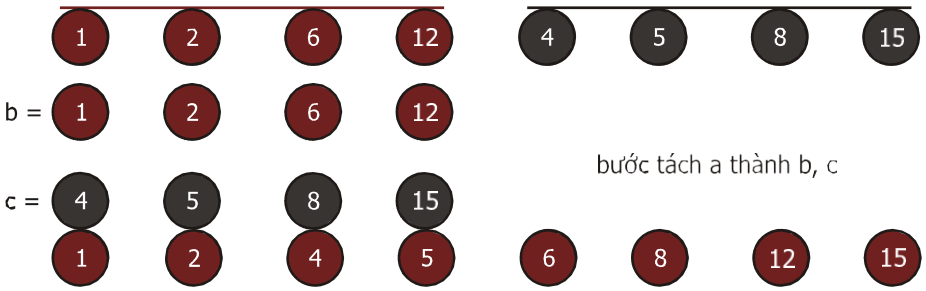
k = 1 trộn



k = 2



k = 4



AUM VIET NAM

7.3.3.3. Đánh giá thuật toán

Về độ phức tạp của thuật toán, ta thấy rằng trong thủ tục Merge, phép toán chủ đạo là thao tác đưa một phần tử vào miền sắp xếp. Còn mỗi lần gọi thủ tục MergeByLenght, tất cả các phần tử được chuyển hoàn toàn sang miền sắp xếp, nên độ phức tạp của thủ tục MergeByLenght là O(n). Thủ tục MergeSort có vòng lặp thực hiện không quá [lgn] lời gọi MergeByLenght bởi biến *len* sẽ được tăng theo cấp số nhân công bội 2. Từ đó suy ra độ phức tạp của MergeSort là O(nlgn) và trong mọi trường hợp độ phức tạp của thuật toán là không đổi.

Chú ý:

- Độ phức tạp của thuật toán MergeSort có tính ổn định tức là độ phức tạp của nó không phụ thuộc vào trạng thái dữ liệu đầu vào cần sắp xếp.
- Nhược điểm của MergeSort là phải dùng thêm một vùng nhớ để chứa một danh sách phụ có chiều dài bằng danh sách cần sắp xếp ban đầu.
- Chúng ta còn có thể lợi dụng được trạng thái dữ liệu vào để khiến MergeSort chạy nhanh hơn đó là ngay từ đầu, chúng ta không coi mỗi khóa là một dãy con của danh sách (dãy con là dãy đã được sắp xếp) mà coi những đoạn đã được sắp xếp trong danh sách là một dãy con. Bởi một danh sách bất kỳ có thể coi là gồm các dãy con được sắp xếp liên tiếp nhau. Khi đó phương pháp trộn này được gọi là **phương pháp trộn tự nhiên**.

7.4. Heap và Heap Sort

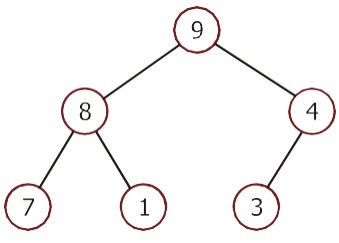
Trong phần này sẽ mô tả một thuật toán sắp xếp là biến thức của cách sắp xếp kiểu chọn đơn giản. Nó dùng một cấu trúc dữ liệu mới gọi là khối (heap) để tổ chức các phần tử của danh sách sao cho có thể thực hiện việc sắp xếp một cách có hiệu quả.

7.4.1. Heap

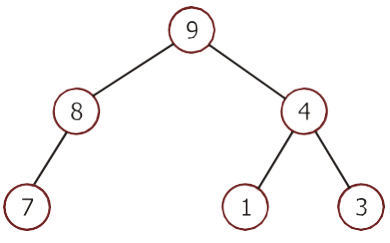
Heap (khối) là một loại cây nhị phân đặc biệt. Nó khác với các cây nhị phân tìm kiếm o hai điểm:

- Khối là **đầy đủ**, nghĩa là các lá trên cây nằm nhiều nhất ở 2 mức phải liên tiếp và các lá ở mức dưới nằm ở những vị trí ”bên trái nhất”.
- Giá trị của mục dữ liệu trong mỗi nút lớn hơn giá trị của những mục dữ liệu ở các con nó (nếu mục dữ liệu là bản ghi thì một trường nào đó trong bản ghi phải thỏa mãn điều kiện này).

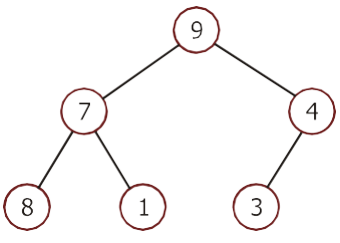
Ví dụ 7.6. Cây nhị phân đầu tiên ở hình a dưới đây là một khối. Cây thứ hai (hình b), không là một khối vì nó không đầy đủ; cây thứ ba (hình c) cũng không là một khối vì dù nó đầy đủ nhưng điều kiện thứ hai về khối lại không thỏa mãn.



Hình a



Hình b



Hình c

AUM VIỆT NAM

Để cài đặt khối, ta có thể dùng một cấu trúc liên kết giống như đối với các cây nhị phân tìm kiếm nhưng để đơn giản và hiệu quả hơn ta dùng mảng để cài đặt khối. Ta đánh số các nút trong khối từ trên (gốc) xuống dưới. Ở mỗi mức, các nút được đánh số từ trái qua phải và lưu dữ liệu ở nút thứ i tại phần tử thứ i của mảng. Mảng Heap được khai báo như sau:

```
#define  HeapLimit …//giới hạn số các nút trong khối
typedef Kieu_du_lieu ElementType;
struct HeapType
   {
     ElementType Array[HeapLimit];
     int m;//số nút trong khối
   } Heap;
```

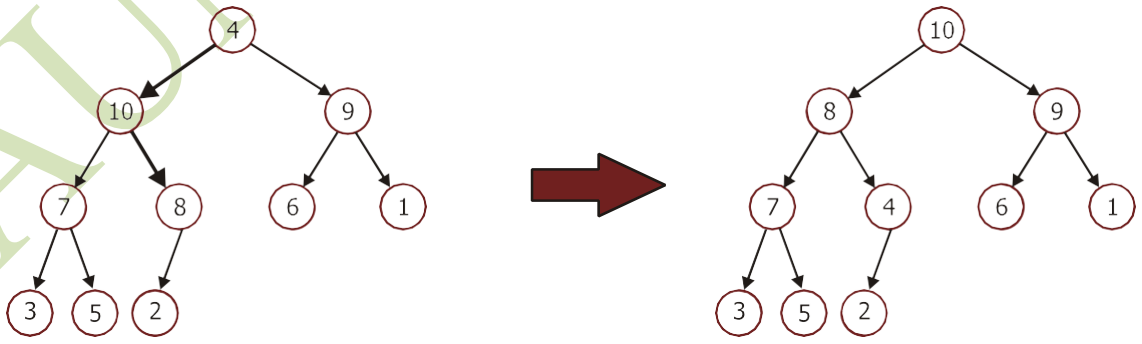
Với cách cài đặt Heap này, các mục trong khối ở ví dụ 7.6 hình a được lưu trữ trong mảng Heap như sau: Heap[1] = 9; Heap[2] = 8; Heap[3] = 4; Heap[4] = 7; Heap[5] = 1; Heap[6] = 3.

Theo bài 5 ta thấy, một danh sách cần sắp xếp được lưu trữ dưới dạng mảng là biểu diễn của một cây nhị phân hoàn chỉnh (đầy đủ). Do đó, vấn đề đặt ra là sắp xếp lại danh sách để nó biểu diễn một khối.

Vì cây nhị phân chỉ có một nút, hiển nhiên là một khối nên để vun một nhánh cây gốc r thành khối, ta có thể coi hai nhánh cây con của nó (nhánh gốc 2r và nhánh gốc 2r + 1) đã là khối rồi và thực hiện thuật toán vun khối từ dưới lên đối với cây.

Để vun khối đối với cây gốc r, tại đó hai nhánh con của r là khối với giả thiết nút r có chứa giá trị V. Từ r, ta đi tới nút con chứa giá trị lớn nhất trong 2 nút con, cho tới khi gặp nút c ở đó mọi nút con của c có giá trị nhỏ hơn hoặc bằng V. Trên đường từ nút r đến nút c, ta đẩy giá trị chứa ở nút con trên đường đi đó lên nút cha và đặt giá V vào nút c.

Hình dưới đây mô tả việc vun một cây nhị phân hoàn chỉnh thành một khối với điều kiện hai nhánh con của nó là khối.



7.4.2. Heap Sort

7.4.2.1. Tư tưởng

Đầu tiên, danh sách A[1..n] được vun thành một khối. Khi đó, phần tử đầu tiên A[1] tương ứng với nút gốc của khối là khóa lớn nhất, ta đảo giá trị của phần tử này cho

AUM VIỆT NAM

phần tử A[n] và không tính tới A[n] nữa. Danh sách A[1…n – 1] còn lại tuy không biểu diễn của một khối nhưng nó biểu diễn cây nhị phân hoàn chỉnh với hai nhanh con: nút thứ 2 và nút thứ 3 (hai nút cây con của nút 1) là khối. Vậy, chỉ cần vun lại một lần, ta được một khối. Đảo giá trị A[1] cho A[n – 1] và tiếp tục cho tới khi khối chỉ còn lại một nút.

7.4.2.2. Nội dung và cách cài đặt thuật toán

Xây dựng thủ tục HeapSort để sắp n phần tử trong mảng A[1], A[2], ..., A[n] theo thứ tự tăng dần. Để xây dựng thuật toán HeapSort ta thực hiện hai thủ tục chính:

- Thủ tục Swapdown chuyển một cây nhị phân đầy đủ lưu tại các vị trí r...n của mảng A với các cây con bên trái và bên phải tạo thành một khối. Thuật toán cài đặt trong thủ tục này như sau:

```
Bước 1:Khởi tạo giá trị
   Done = 0;
   C = 2 * r;
Bước 2:
  while ((Done != 0)&&(C <= n - 1))//tìm con lớn nhất
   {
      if (C < n - 1)
         if (A[C] < A[C + 1])
           C = C + 1;
             if (A[r] < A[C])
               {
                  Interchange(A[r], A[C]);
                  //hàm để hoán vị A[r]với A[C]
                  r = C;
                  C = C* 2;
               }
             else
                Done = 1;
   }
```

Sử dụng thủ tục HeapiFy nêu dưới đây để chuyển một cây nhị phân đầy đủ lưu tại các vị trí từ 1...n của mảng A thành một khối. Áp dụng thủ tục SwapDown cho cây được lưu trữ trong các vị trí từ r đến n của mảng A

Khi đó thuật toán HeapSortđược thể hiện như sau:

```
Bước 1:
   HeapiFy(A);//hàm chuyển cây thành khối
Bước 2:
   for (int i = n - 1; i >= 1; i++)
       {
             Interchange(A[0],A[i]);//hàm hoán vị A[0] và A[i]
             SwapDown(A, 0);
        }
```

AUM VIỆT NAM

Cài đặt thuật toán HeapSort

```
void Interchange(ElementType X,ElementType Y)
  {
        ElementType Temp;
        Temp = X;
        X = Y;
        Y = Temp;
  }
void SwapDown(HeapType Heap,int r)
{
        int Child,Done; Done
        = 0;
        Child = 2 * r;
        while ((Done != 0)&&(Child <= Heap.m – 1))
           {
                  if (Child < Heap.m – 1)
                        if (Heap.Array[Child] < Heap.Array[Child + 1]) Child = Child + 1;
                         if (Heap.Array[r] < Heap.Array[Child])
                              {
                                 Interchange(Heap.Array[r],Heap.Array[Child]); r = Child;
                                                                    Child = Child * 2;
                              }
                       else
                           Done = 1;
           }
}
void HeapiFy(HeapType Heap)
{
      int r;
      for (r = Heap.m/2 – 1; r >= 0; r—)
                SwapDown(Heap, r);

}
void HeapSort(HeapType A)
{
    HeapiFy(A);
    for (int i = A.m – 1; i >= 1; i++)
          {
                Interchange(A.Array[0],A.Array[i]); SwapDown(A,
                0);
          }
}
```

AUM VIỆT NAM

7.4.2.3. Đánh giá thuật toán

Ta nhận thấy độ phức tạp của HeapSort phụ thuộc chủ yếu vào độ phức tạp của các thuật toán SwapDown và HeapiFy. Trong SwapDown, số các mục trong danh sách con xét trong mỗi giai đoạn bằng một nửa số các mục trong danh sách con ở giai đoạn trước. Như vậy, bằng phân tích tương tự như đối với các cây nhị phân tìm kiếm, dễ thấy, độ phức tạp của thuật toán này trong trường hợp xấu nhất là $O(\log_2 n)$. Và thuật toán HeapiFy thực hiện SwapDown n/2 lần, độ phức tạp của nó trong trường hợp xấu nhất là $O(n\log_2 n)$. HeapSort chỉ thực hiện HeapiFy một lần và SwapDown n – 1 lần; từ đó suy ra, độ phức tạp của HeapSorttrong trường hợp xấu nhất là $O(n\log_2 n)$.

AUM VIỆT NAM

TÓM LƯỢC CUỐI BÀI

- Cùng một mục đích sắp xếp như nhau, nhưng có nhiều phương pháp giải quyết khác nhau. Nếu chỉ dựa vào thời gian tính toán của thuật toán đo được trong một ví dụ cụ thể mà đánh giá thuật toán này tốt hơn thuật toán kia về mọi mặt là chưa đủ. Việc chọn một thuật toán sắp xếp phù hợp với từng yêu cầu, từng điều kiện cụ thể là kỹ năng của người lập trình.
- Những thuật toán có độ phức tạp $O(n^2)$ như sắp xếp kiểu chọn, sắp xếp kiểu chèn hay sắp xếp đổi chỗ thì chỉ nên áp dụng cho các chương trình sắp xếp có ít lần sắp xếp và với kích thước n nhỏ. Về tốc độ, Bubble Sort luôn đứng sau cùng nhưng nó hết sức đơn giản cho người mới lập trình. Thuật toán chèn tỏ ra nhanh hơn các thuật toán còn lại và có tính ổn định; mã lệnh dễ nhớ.
- Những thuật toán sắp xếp chia để trị như Quick Sort, Merge Sort hay Heap Sort là những thuật toán sắp xếp tổng quát, dãy khóa thuộc kiểu dữ liệu có thứ tự nào cũng có thể áp dụng được. Quick Sort gặp nhược điểm trong trường hợp suy biến (dù trường hợp này là rất nhỏ), Merge Sort đòi hỏi thêm không gian nhớ phụ, còn HeapSort thì mã lệnh hơi phức tạp và khó nhớ. Tuy nhiên, những nhược điểm này là quá nhỏ. Ưu điểm chung của chúng là nhanh so với các thuật toán khác.

AUM VIỆT NAM

BÀI TẬP

1. Trình bày tư tưởng của các thuật toán sắp xếp?

2. Trong các thuật toán sắp xếp bạn thích nhất là thuật toán nào? Thuật toán nào bạn không thích nhất? Tại sao?

3. Dùng các biểu đồ dạng cây nhị phân đầy đủ để mô tả áp dụng thuật toán chuyển một cây nhị phân đầy đủ sang một khối trong HeapSortcho các danh sách sau:

a) 7, 1, 6, 5, 4, 2, 3

b) 1, 7, 2, 6, 3, 5, 4

c) 7, 6, 5, 4, 3, 2, 1

d) 1, 2, 3, 4, 5, 6, 7

4. Nhập vào một dãy số nguyên gồm n phần tử. Viết chương trình dưới dạng menu có sử dụng tất cả các thuật toán đã học để sắp xếp dãy số theo thứ tự không giảm.

AUM VIỆT NAM